

Secure boot, Verified boot, U-boot, And other stuff And RAUC

Agenda

(roughly)

- Goals, Definitions, ~~Crypto~~ Recap
- i.MX HABv4
- Bootloaders & U-Boot
- FIT-Image
- DM-Verity
- Encryption?
- A/B-Update with RAUC

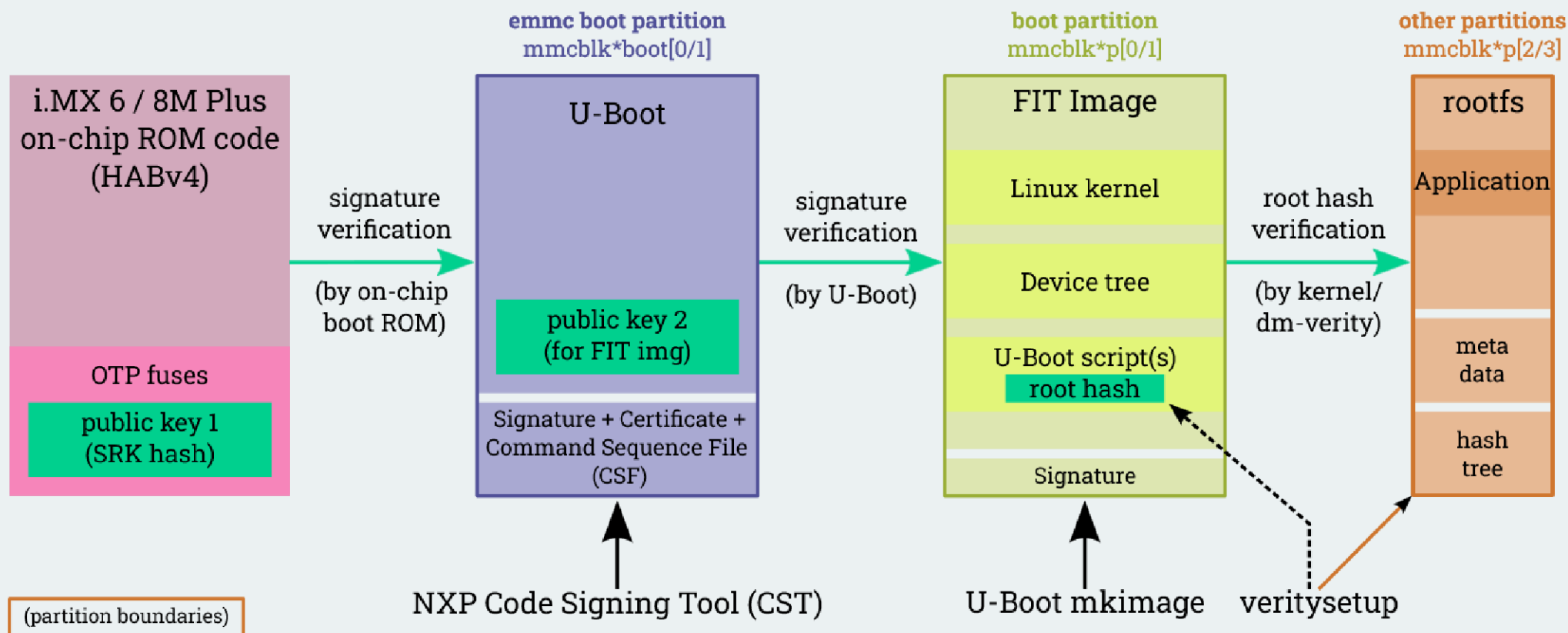
Motivation (of this talk)

- “Demystify” embedded linux boot process
- “Demystify” secure boot
- Kick-start the Munich Embedded Scene!
 - Join the Matrix channel!

Goals

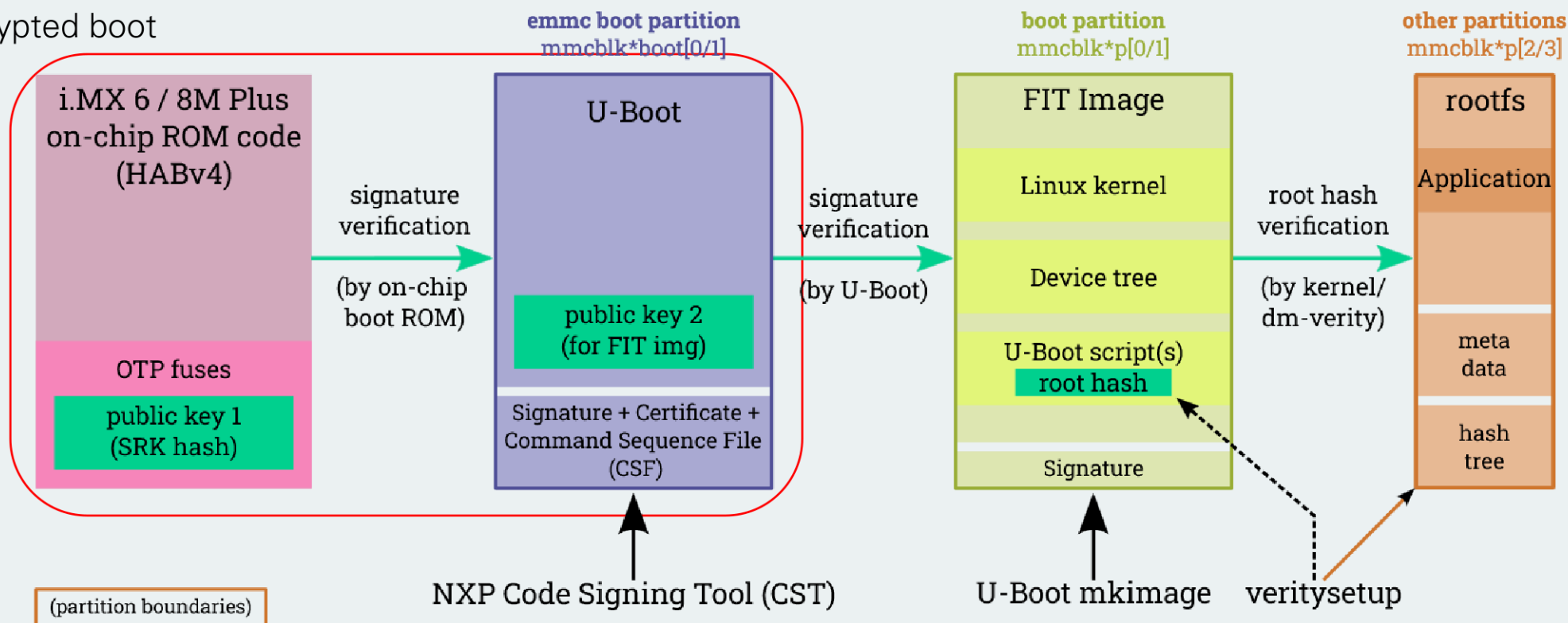
- Allow only execution of vendor-authorized code (bootloader, kernel, application, etc.)
 - Helps to protect intellectual property and device secrets
 - Any modification is detected immediately and causes a controlled failure
- Guarantee system integrity from reset to user space
 - No execution gaps where unsigned or modified code can run
- Allow Software Updates (with rollback) (-> rauc)
- As easy as possible, as complex as needed
- Use off-the-shelf stuff whenever possible
- Project environment:
 - Yocto, u-boot, i.MX6 / i.MX8m (-> HABv4, not AHAB)

Overview



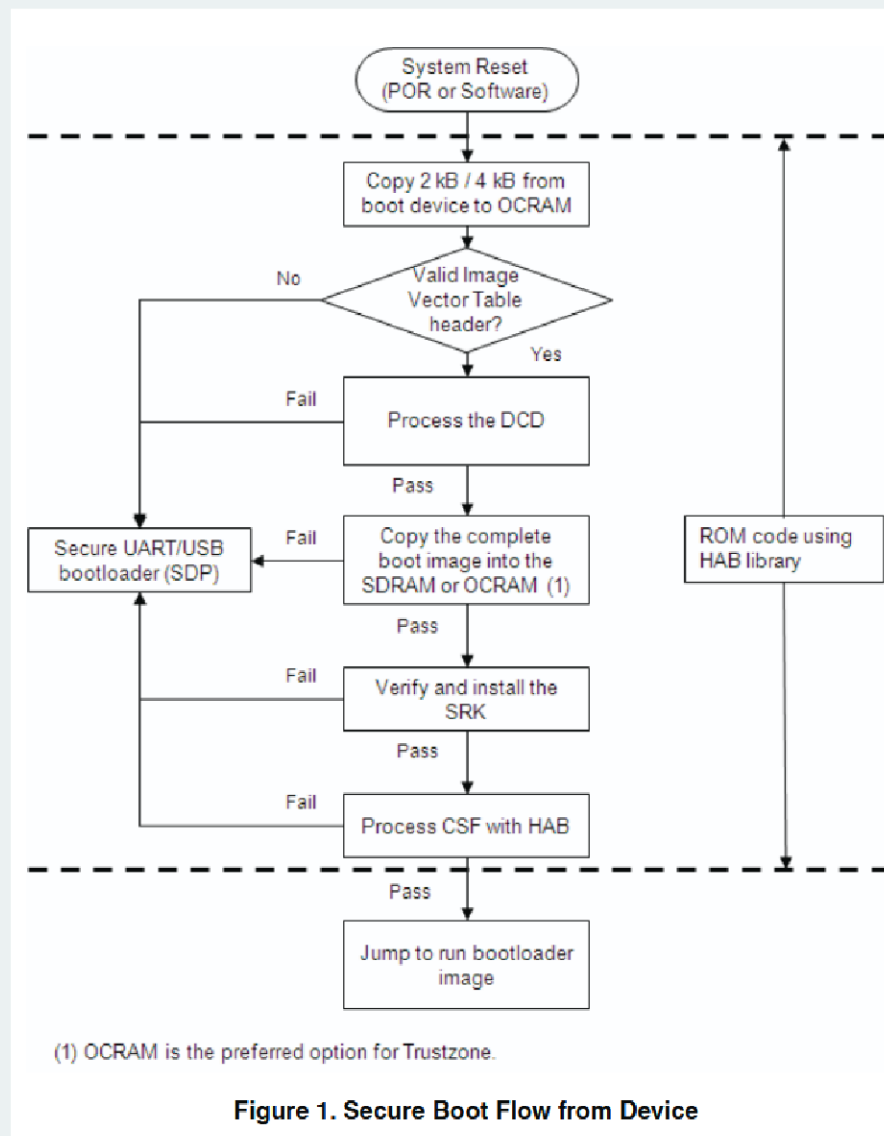
HABv4

- NXP on-chip Boot ROM
- Verifies signed boot container with Public Key Hash in eFuses
- Could also verify more than only the bootloader
- Supports encrypted boot



HABv4

- NXP on-chip Boot ROM
- Verifies signed boot container with Public Key Hash in eFuses
- Could also verify more than only the bootloader
- Supports encrypted boot



Source: NXP

HABv4

- Generate 4 Super Root Keys (SRK)
- SRK Table
- SRK (Table) Hash
- Burn SRK Hash into Fuses
- Sign u-boot with 1 SRK
- Generate Command Sequence File
 - Contains the full SRK Table
- (Generate Device Configuration Data (DCD))
- Build boot image

 csf_uboot_fast_authentication.txt  628 B

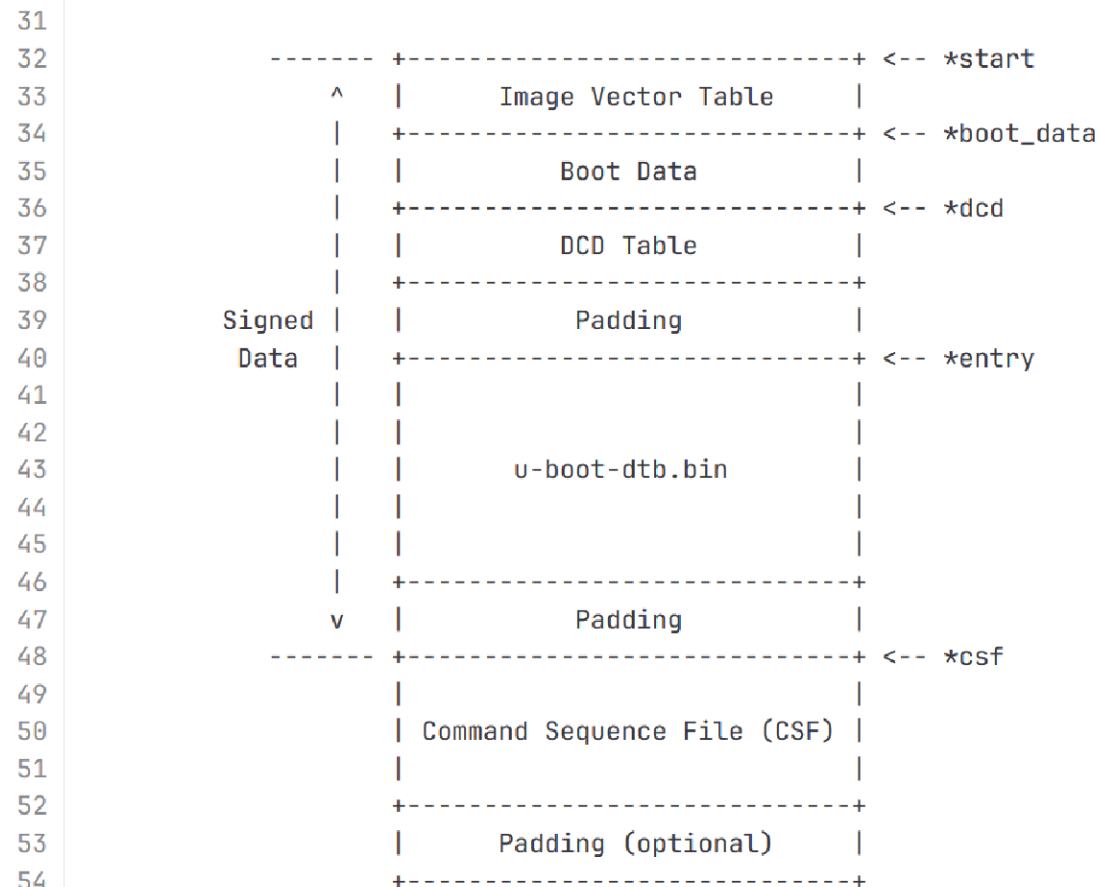
```
1 [Header]
2     Version = 4.2
3     Hash Algorithm = sha256
4     Engine Configuration = 0
5     Certificate Format = X509
6     Signature Format = CMS
7     Engine = CAAM
8
9 [Install SRK]
10     # Index of the key location in the SRK table to be installed
11     File = "../crts/SRK_1_2_3_4_table.bin"
12     Source index = 0
13
14 [Install NOCAK]
15     File = "../crts/SRK1_sha256_2048_65537_v3_usr.crt.pem"
16
17 [Authenticate CSF]
18
19 [Authenticate Data]
20     # Key slot index 0 used to authenticate the image data
21     Verification index = 0
22     # Authenticate Start Address, Offset, Length and file
23     Blocks = 0x877ff400 0x00000000 0x0009ec00 "u-boot-dtb.imx"
24
```

HABv4

- Generate 4 Super Root Keys (SRK)
- SRK Table
- SRK (Table) Hash
- Burn SRK Hash into Fuses
- Sign u-boot with 1 SRK
- Generate Command Sequence File
 - Contains the full SRK Table
- (Generate Device Configuration Data (DCD))
- Build boot image

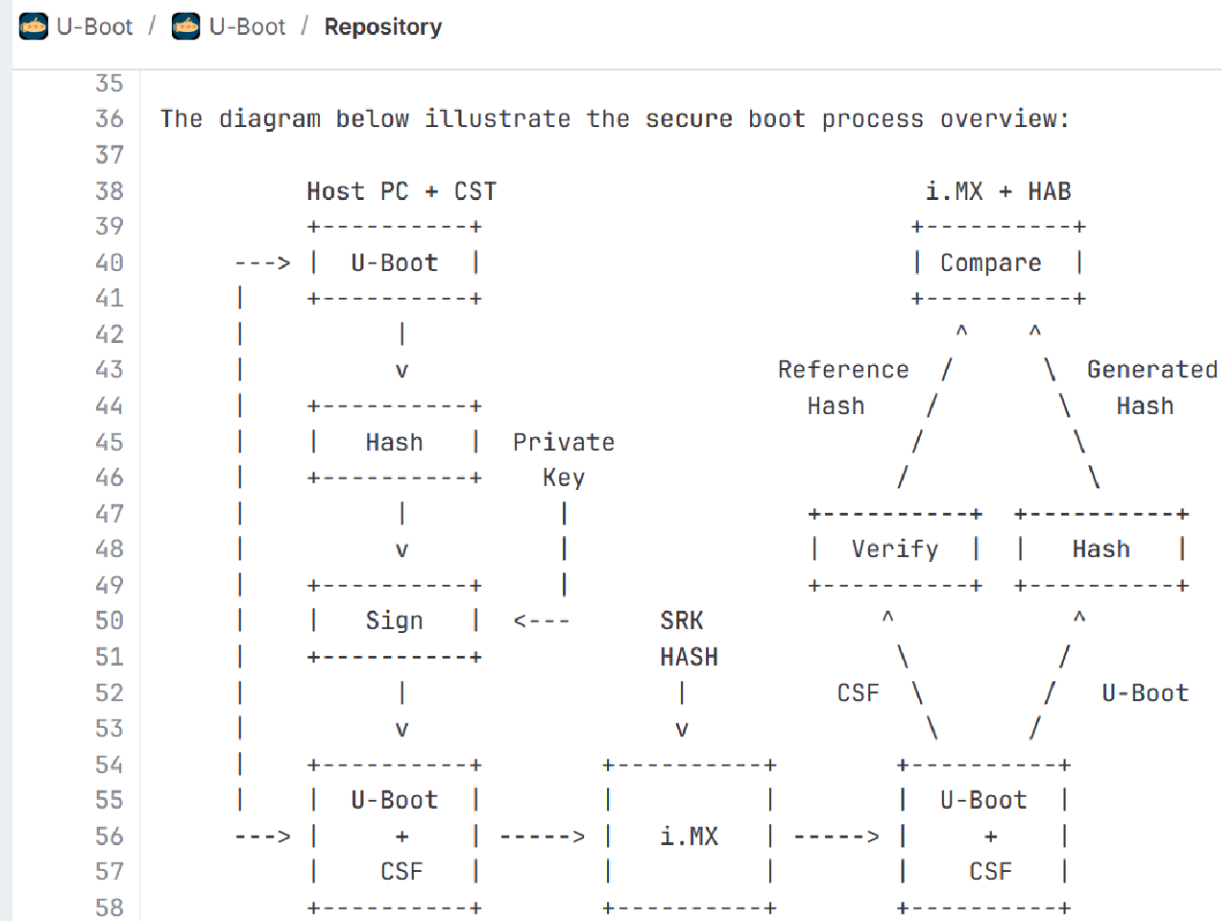
U-Boot / U-Boot / Repository

30 The diagram below illustrate a signed u-boot-dtb.imx image layout:



HABv4

- Generate 4 Super Root Keys (SRK)
- SRK Table
- SRK (Table) Hash
- Burn SRK Hash into Fuses
- Sign u-boot with 1 SRK
- Generate Command Sequence File
 - Contains the full SRK Table
- (Generate Device Configuration Data (DCD))
- Build boot image



U-Boot

- Problem: Linux Kernel needs various peripherals (and memory!) to be set up in advance
- Solution: Bootloader!
- Usually 2 Steps:
 - SPL (Secondary Program Loader)
 - Loaded by Boot ROM into tiny on-chip RAM (OCRAM)
 - Main job: set up DRAM and load “U-boot proper”
 - Not strictly needed here because Boot ROM can set up DRAM with Device Configuration Data (DCD)
 - U-boot proper
 - Set up Screen, USB, Network, etc...
 - Provides debug console
 - Load fitImage and boot correct kernel with correct parameters
- Single binary running bare-metal (nice for HW debugging!)
- Scriptable
- Support for Environment variables
- Configurable with Kconfig
- Provides Fastboot Interface to flash emmc
- **Beware of vendor downstream forks!:** uboot-imx, uboot-fslc
- Alternative: barebox

FIT-Image

- “new” container format, replaces ulmage
- “Flattened Image Tree”, basically a device tree
- Like dts/dtb:
 - *.its = Image Tree Source
 - *.itb = Image Tree Blob
 - dtc for (de)compiling
- Can contain
 - Configuration data
 - Binaries
 - Kernel, DTB, Initramfs, U-boot scripts, Firmware (optee, etc)
- Can be signed!
 - public key in u-boot, private key in HSM (pkcs#11)
 - -> U-Boot FIT Signature Verification

```
1 /dts-v1/;
2
3 / {
4     description = "Kernel fitImage for development";
5     #address-cells = <1>;
6
7     images {
8         kernel-1 {
9             description = "Linux kernel";
10            data = /incbin/("linux.bin");
11            type = "kernel";
12            arch = "arm";
13            os = "linux";
14            compression = "none";
15            load = <0x10008000>;
16            entry = <0x10008000>;
17            hash-1 {
18                algo = "sha512";
19            };
20        };
21        fdt-imx6dl-demo {
22            description = "Flattened Device Tree blob";
23            data = /incbin/("arch/arm/boot/dts/nxp/imx/imx6dl-demo.dtb");
24            type = "flat_dt";
25            arch = "arm";
26            compression = "none";
27            load = <0x18000000>;
28            hash-1 {
29                algo = "sha512";
30            };
31        };
32    };
33
34    configurations {
35        default = "conf-imx6dl-demo.dtb";
36        conf-imx6dl-demo.dtb {
37            description = "1 Linux kernel, FDT blob";
38
39            kernel = "kernel-1";
40            fdt = "fdt-imx6dl-demo";
41
42            hash-1 {
43                algo = "sha512";
44            };
45            signature-1 {
46                algo = "sha512,rsa4096";
47                key-name-hint = "FIT-4096";
48                padding = "pkcs-1.5";
49                sign-images = "kernel", "fdt";
50            };
51        };
52    };
53 };
```

FIT-Image

- “new” container format, replaces ulmage
- “Flattened Image Tree”, basically a device tree
- Like dts/dtb:

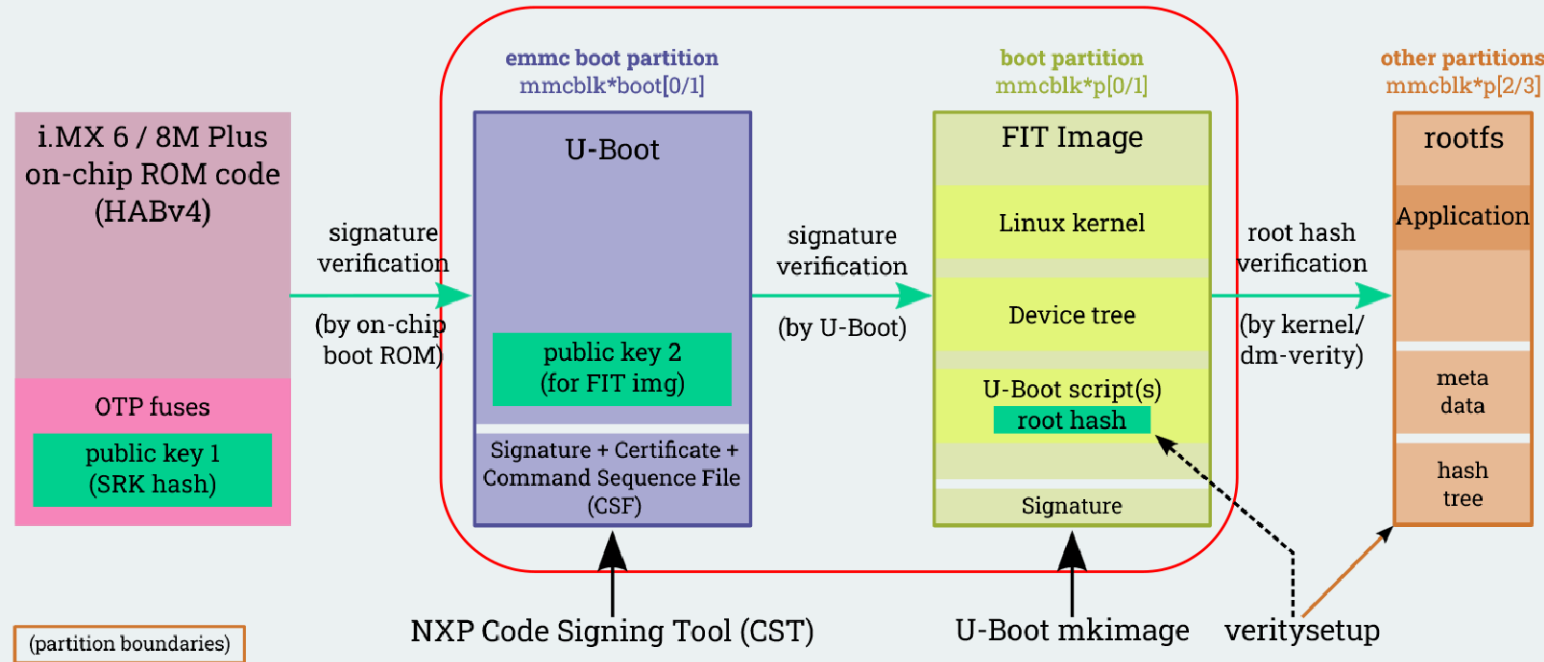
- *.its = Image Tree Source
- *.itb = Image Tree Blob
- dtc for (de)compiling

- Can contain

- Configuration data
- Binaries
 - Kernel, DTB, Initramfs, U-boot scripts, Firmware (optee, etc)

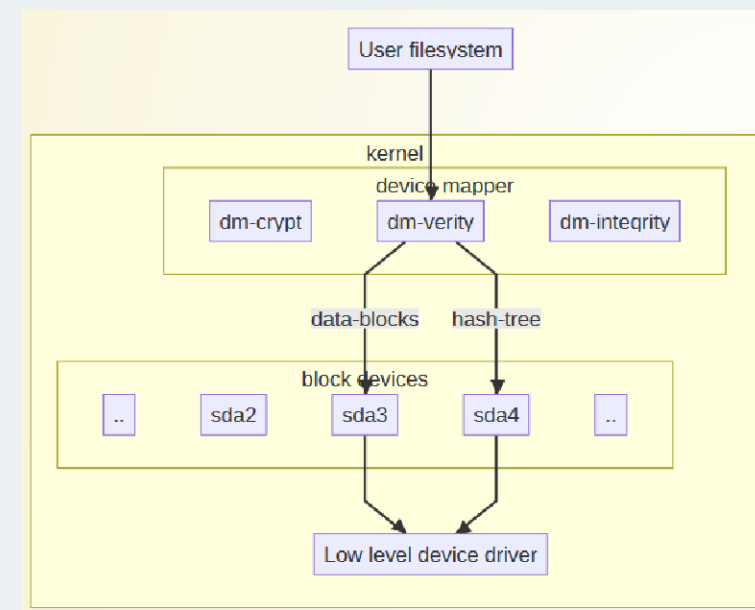
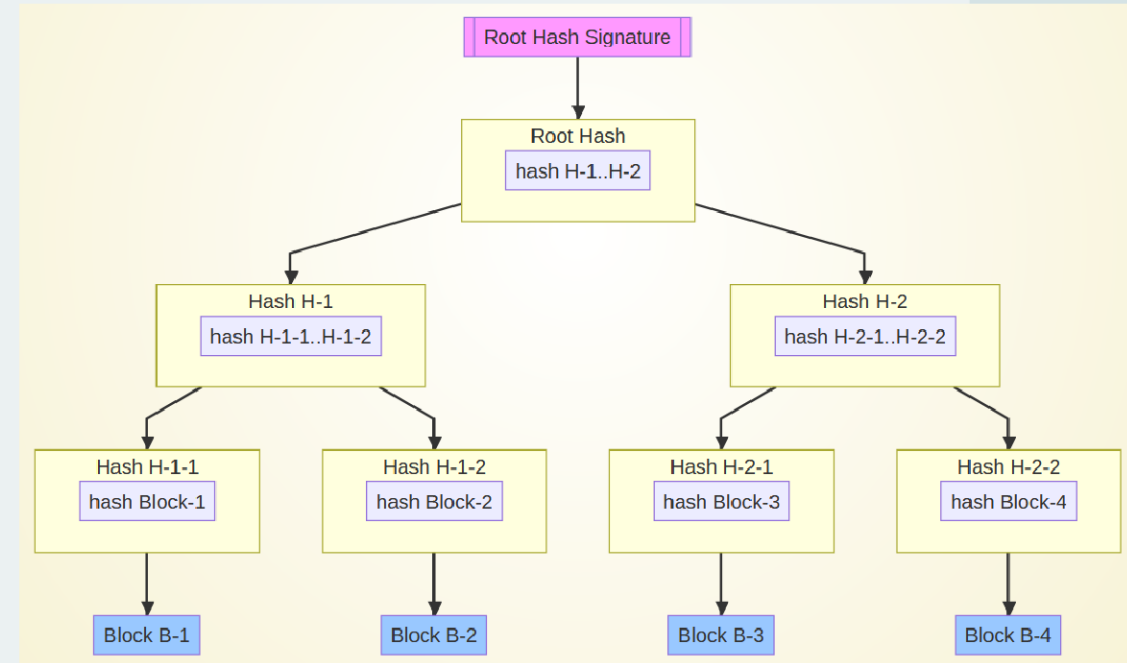
- Can be signed!

- public key in u-boot, private key in HSM (pkcs#11)
- -> U-Boot FIT Signature Verification



dm-verity

- Linux Device Mapper – Verity Mapper
- Transparently checks every block on access
- Hash Tree on dedicated partition
- Implicitly Read-Only!
- Works without initramfs
- May contain Forward Error Correction
- Root Hash may be signed
(still hacky. We do not use this, but pass the root hash directly to the kernel.
This ties the rootFS to the fitImage)
- Check out FOSDEM Talk from Frank:
https://archive.fosdem.org/2023/schedule/event/image_linux_secureboot_dmverity/



dm-verity

- Linux Device Mapper – Verity Mapper
- Transparently checks every block on access
- Hash Tree on dedicated partition
- Implicitly Read-Only!
- Works without initramfs
- May contain Forward Error Correction
- Root Hash may be signed
(still hacky. We do not use this, but pass the root hash directly to the kernel.
This ties the rootFS to the fitImage)
- Check out FOSDEM Talk from Frank:
https://archive.fosdem.org/2023/schedule/event/image_linux_secureboot_dmverity/

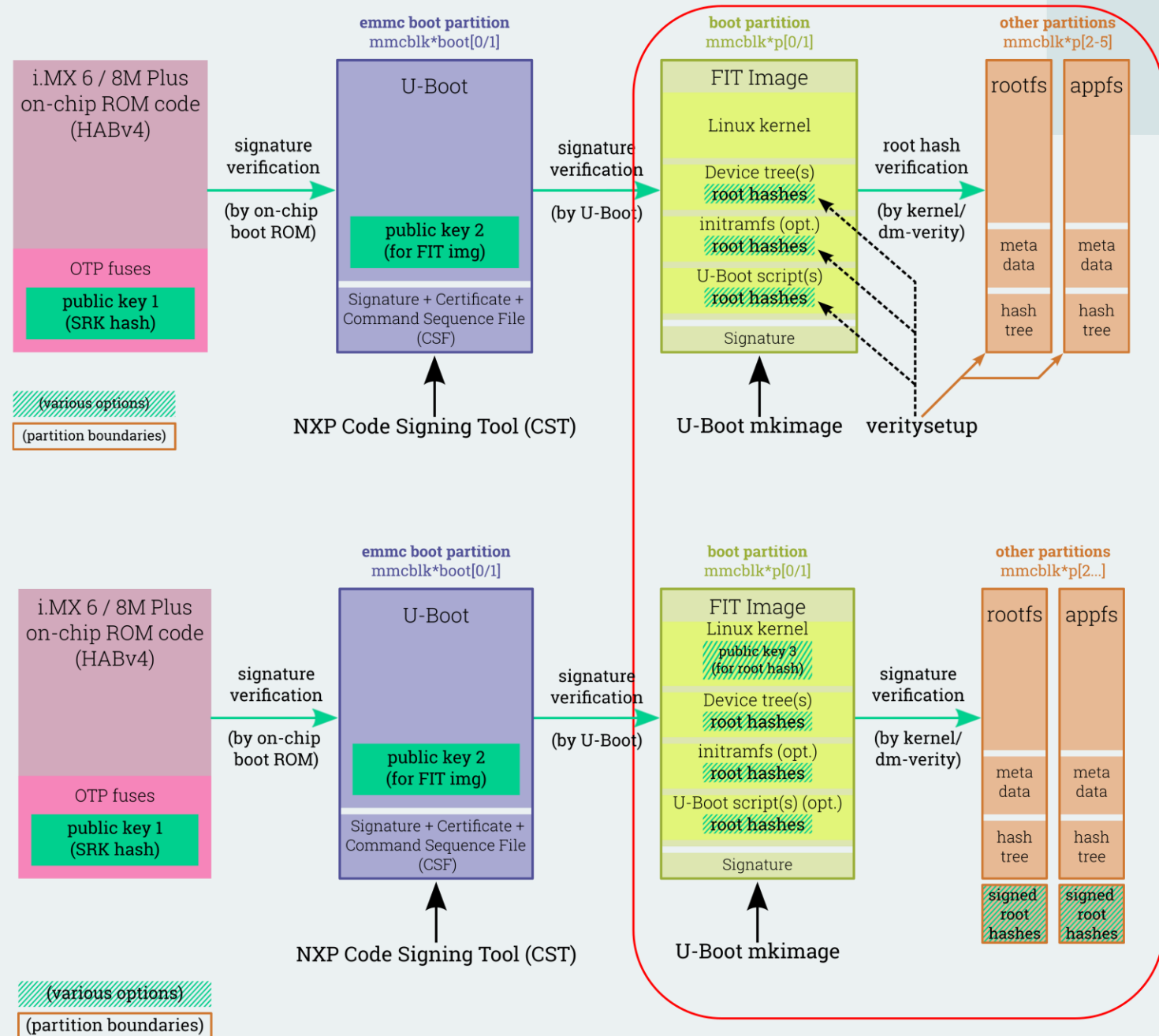
Kernel command line example for signed root hash:

```
1 console=ttyAMA0,115200 ro rootwait
2 root=/dev/dm-0
3 systemd.volatility=overlay
4 dm-mod.create="verity,,,ro,0 122880 verity 1
5 /dev/sda4 /dev/sda5 1024 4096
6 61440 1 sha256
7 a100bab0d2e49b665bb18a0ee202b0fb78d084e0d6cfe3b115b6e3908b8ac1
8 5027c03524f90fdf6e71f623ce059591a0aa3b48a39807826244c5f5e1db3a
9 3 ignore_zero_blocks
10 root_hash_sig_hex
11 3081df06092a864886f70d010702a081d13081ce020101310f300d06096086
12 dm_verity.require_signatures=1 rauc.slot=B
```

Image: <https://frehberg.com/2023/02/dm-verity-presentation-slides/>

dm-verity

- Various ways to store and pass the root hash to the kernel
- Changing something in rootFS always means replacing the fitImage...
- ... unless signed root hashes are used – which is tricky but feasible. (can be useful for replacing only the application on a dedicated filesystem)
- Mixed/Hybrid approaches are possible.

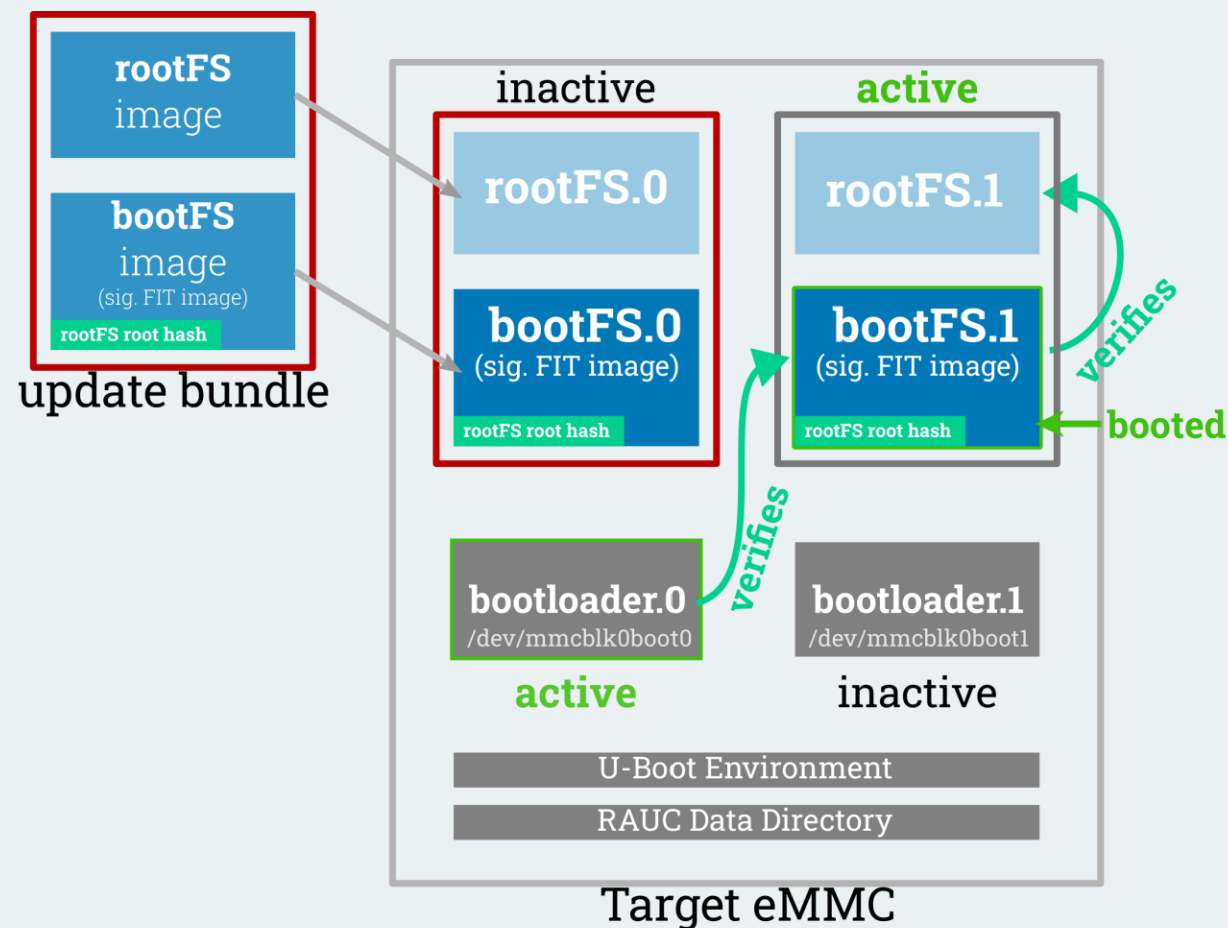


Encryption?

- HABv4 Encrypted Boot
 - Could encrypt the bootloader and other data
- (Encrypted FIT-image should also be possible)
- NXP CAAM & DM-Crypt (& DM-Verity / DM-Integrity)
- Various possibilities:
 - Encrypted Container
 - Encrypted Partition
 - Encrypted Rootfs
- CAAM Key can't be extracted!

RAUC: Robust Auto Update Controller

- based on partition images
- Supports various encryption/authentication options
- Various A/B/Recovery and Rollback options possible
-> Flexible and Failsafe
- Alternatives: SWUpdate, Mender,...



Sources and thanks

- <https://rauc.readthedocs.io/>
- <https://pengutronix.de/en/blog/2024-11-19-generic-fit-image-support-for-meta-oe.html>
- <https://docs.u-boot.org/en/latest/>
- <https://docs.u-boot.org/en/stable/usage/fit/howto.html>
- <https://source.denx.de/u-boot/u-boot/-/tree/master/doc/imx/habv4>
- <https://www.devicetree.org/>
- https://archive.fosdem.org/2023/schedule/event/image_linux_secureboot_dmverity/
- <https://fosdem.org/2026/schedule/event/SZRFKD-barebox-single-image-secure-boot/>
- https://trac.gateworks.com/wiki/secure_boot
- Various NXP Application Notes:
AN4581, AN12714, UG10106, IMX6HABUG, IMX6DQ6SDLSRM, RM00298
- Thanks to the Munich Embedded Community
- Questions and Feedback: matthias_munichembedded@<lastname>.eu